

## INFORMATION ZUR VORLESUNG

# REST – Distributed Parcel System

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge  
Thema Eine REST-API entwerfen und implementieren  
Dozent Dr. Ulrich Winkler  
Semester 5. Semester · Informationstechnik

## REST – Distributed Parcel System

Übungsreihe zur REST-Vorlesung (5. Semester, Informationstechnik, DHSN Dresden). Sie entwerfen die API eines kleinen Paketdienstes und implementieren sie in Python. Die Übungen bauen aufeinander auf:

1. `01_parcel_service/aufgaben.md` : REST-API und dateibasierte Paketablage.
2. `02_atomar_file_ops/aufgaben.md` : atomare Dateioperationen und Absturzfenster.
3. `03_transactional_outbox/aufgaben.md` : PNG-Upload in temporäre Dateien, SQLite-Outbox und Einordnen durch einen separaten File-Service.

Der rote Faden ist nicht CRUD. Es geht um die Eigenschaften, die REST in einem verteilten System überhaupt erst nützlich machen: idempotente Methoden, bedingte Requests, Nebenläufigkeit, Caching und definiertes Verhalten, wenn das Netz unzuverlässig ist.

### In Übung 01 und 02: keine Datenbank – und das ist der Punkt

Ihre Paket- und Tracking-Daten liegen in **Dateien**, nicht in einem DBMS. Das ist eine bewusste Entscheidung: Mit einer relationalen Datenbank wären die interessanten Aufgaben der ersten Übungen in je einer Zeile SQL erledigt.

Mit Dateien müssen Sie sie selbst bauen. Sie werden dabei auf genau die Probleme stoßen, die ein Datenbanksystem intern löst:

| Was eine DB Ihnen schenkt | Was Sie hier selbst brauchen                                    |
|---------------------------|-----------------------------------------------------------------|
| UNIQUE -Constraint        | atomares Anlegen ( <code>O_CREAT</code>   <code>O_EXCL</code> ) |
| SELECT ... FOR UPDATE     | Dateisperre ( <code>fcntl.flock</code> )                        |
| atomarer Commit           | in Temp-Datei schreiben, dann <code>os.replace</code>           |
| Versionsspalte            | ein <code>version</code> -Feld, das Sie selbst pflegen          |

Die Umgebungsvariable `DATA_DIR` zeigt auf das Datenverzeichnis ( `loesung/data` ). Lesen Sie den Pfad daraus, statt ihn in den Code zu schreiben. Der Devcontainer legt die Laufzeitverzeichnisse an; Ihr Code sollte sie auch ohne Devcontainer bei Bedarf anlegen. In Übung 03 bleibt diese Dateiablage erhalten. Nur Bild-Metadaten und die Outbox liegen gemeinsam in einer SQLite-Datei unter `$DATA_DIR`. Der separate File-Service verwendet `$FILE_DATA_DIR` für die endgültigen Bilddateien, etwa `loesung/file-data`. `$STAGING_DIR` (etwa `loesung/staging`) ist ein gemeinsames temporäres Verzeichnis für beide Dienste auf demselben Dateisystem.

## Was der Container mitbringt – und was nicht

| Bestandteil                       | wofür                                     |
|-----------------------------------|-------------------------------------------|
| Python 3.12                       | Laufzeit                                  |
| uv                                | Paketmanager                              |
| curl, httpie, jq                  | HTTP-Requests von Hand stellen            |
| sqlite3                           | SQLite-Outbox untersuchen (Übung 03)      |
| uuidgen                           | Idempotency-Keys erzeugen                 |
| REST Client (VS-Code-Erweiterung) | .http -Dateien direkt im Editor ausführen |
| Thunder Client, Postman           | Klick-Oberflächen mit Historie            |

`loesung/` ist Ihr Arbeitsbereich für das Python-Projekt. Der Devcontainer installiert beim Erstellen die Abhängigkeiten aus `loesung/pyproject.toml` und `loesung/uv.lock`: FastAPI, Uvicorn, `python-multipart` für den Bild-Upload, `httpx` für den Relay und `pytest`. Für die Dateiablage brauchen Sie keine zusätzliche Bibliothek – `json`, `os`, `pathlib`, `fcntl` und `hashlib` aus der Standardbibliothek reichen aus. Übung 03 verwendet außerdem Pythons `sqlite3`-Modul.

### Uvicorn

Uvicorn ist ein ASGI-Webserver für Python. Er nimmt HTTP-Verbindungen entgegen und führt darüber Python-Webanwendungen aus, insbesondere Frameworks wie FastAPI und Starlette.

Die folgenden Befehle führen Sie im Verzeichnis `loesung/` aus, nachdem Sie Ihre Anwendung als `main.py` angelegt haben.

ASGI steht für Asynchronous Server Gateway Interface. Es ist eine Schnittstelle/Standard zwischen einem Python-Webserver und einer Python-Webanwendung.

```
# Entwicklung – ein Prozess, lädt bei Änderungen neu
uv run uvicorn main:app --reload --host 0.0.0.0 --port 8000

# Für echte Nebenläufigkeit vier Prozesse
uv run uvicorn main:app --workers 4 --host 0.0.0.0 --port 8000
```

Nicht beides zusammen angeben: Uvicorn bricht dann nicht ab, sondern schreibt nur `WARNING: "workers" flag is ignored when reloading is enabled.` und läuft mit einem einzigen Prozess weiter.

### Wie Sie arbeiten

#### GitHub Codespaces (empfohlen – nichts zu installieren)

1. **Code - Codespaces - Create codespace on main.**
2. Warten, bis der Container gebaut ist (beim ersten Mal einige Minuten).
3. Terminal öffnen und loslegen.

#### Lokal in VS Code

Docker und die Erweiterung **Dev Containers** installieren, Repo klonen, Ordner öffnen und **Reopen in Container** wählen. Unter Windows brauchen Sie WSL2 mit Docker Desktop; ohne das nehmen Sie bitte Codespaces.

**Hinweis für Windows-Nutzer, die lokal arbeiten:** `fcntl` gibt es nur auf POSIX-Systemen. Im Dev-Container (Linux) ist das kein Problem – arbeiten Sie also im Container und nicht daneben.

## Aufbau

```
.devcontainer/          Container und Werkzeuge
uebungen/
├─ 01_parcel_service/    REST-API und dateibasierte Paketablage
├─ 02_atomar_file_ops/   atomare Dateioperationen
└─ 03_transactional_outbox/ SQLite-Outbox und separater File-Service
loesung/                 Ihr Python-Projekt
├─ data/                 Paketablage und SQLite zur Laufzeit (nicht
committen)
├─ staging/              temporäre Bilddateien (nicht committen)
└─ file-data/            einsortierte Bilddateien (nicht committen)
```